

DATA PRIVACY

LECTURER: PROF. DR. FATIH TURKMEN

TAs: ALI SATVATY AND PABLO SORRENTINO

Assignment 1

Deadline: September 25, 2026

Please read carefully the [Assignment Guidelines](#) document regarding the rules, grading criteria, etc.

September 11, 2026



UNIVERSITY OF GRONINGEN

1 k-anonymity (25p)

Assume an organization is conducting a survey about political tendencies of voters in an election. The survey consists of the Age, Zip Code, Profession and (political) Alignment of a subject where the political alignment information is sensitive. The organization decides to make this data public after some anonymization, namely suppression and generalization. The original data is the following obtained from the survey is provided in Table 1.

Age	Zip Code	Profession	Alignment
25	56601	Engineer	PartyX
34	56602	Doctor	PartyY
66	56601	Teacher	PartyX
24	56604	Engineer	PartyZ
47	56603	Doctor	PartyZ
51	56602	Student	PartyY
22	56603	Student	PartyX
28	56601	Doctor	PartyY
31	56603	Engineer	PartyZ
34	56602	Student	PartyX

Table 1: Political tendencies (related to question 1)

- (a) Identify quasi-identifiers (equivalence classes) and explain how you obtained it.
- (b) By employing suppression on any combination of columns/rows, make the released table 2-anonymous (keep the data in the table as much as possible).
- (c) By employing generalization on any combination of columns/rows, make the released table 2-anonymous (keep the data in the table as much as possible).

Note:

- CSV version of this file is available in Brightspace (as part of the assignment).
- You can refer to Wikipedia¹ and ² for information regarding the definition of quasi-identifiers and K-anonymity and how to apply it.
- You can assume that suppression and generalization can be applied even at the cell level.

¹<https://en.wikipedia.org/wiki/K-anonymity>

²<https://elf11.github.io/2017/05/22/kanonymity.html>

2 Differential Privacy (25p)

Explain the intuition behind ϵ -differential privacy from the following points:

- (a) What type of problem it tries to solve and what it adds to traditional anonymization techniques.
- (b) How it works in general terms. You can use simple examples to explain the concepts (e.g., function sensitivity level).
- (c) Given Table 2 (adapted from the UCI Machine Learning Repo), the adversary is allowed to count the number of people with income more than X (through querying) but we want to do this in a privacy-preserving way, i.e., the result is returned with a noise.
 1. What is the level of (local) sensitivity of the function we are considering?
 2. Show the result of applying Laplace noise³ with the results of the count query, and summarize your observations.

Note: CSV version of this file is available in Brightspace (as part of the assignment).

Age	Workclass	Education	Occupation	Ethnicity	Gender	Hours / Week	Native-country	Income
39	State-gov	Bachelors	Adm-clerical	White	Male	40	United-States	80000
50	Self-emp-not-inc	Bachelors	Exec-managerial	White	Male	13	United-States	45000
38	Private	HS-grad	Handlers-cleaners	White	Male	40	United-States	50000
53	Private	11th	Handlers-cleaners	Black	Male	40	United-States	60000
28	Private	Bachelors	Prof-specialty	Black	Female	40	Cuba	70000
37	Private	Masters	Exec-managerial	White	Female	40	United-States	45000
49	Private	9th	Other-service	Black	Female	16	Jamaica	40000
52	Self-emp-not-inc	HS-grad	Exec-managerial	White	Male	45	United-States	100000
31	Private	Masters	Prof-specialty	White	Female	50	United-States	30000
42	Private	Bachelors	Exec-managerial	White	Male	40	United-States	90000
37	Private	Some-college	Exec-managerial	Black	Male	80	United-States	50000

Table 2: People and Their incomes (related to question 2)

³You are free to choose the ϵ value but you can also inspire from here: <https://blog.openmined.org/choosing-epsilon/>

3 Membership Inference on Language Models (50+20p)

Target model	EleutherAI/pythia-1b
Group size	2–3 students
Your attack ID	group_number % 4 (see Section 3.3)
Deliverable	report, max. 6 pages excluding appendix, plus code and saved results in a Jupyter Notebook

The subsections below state *what you have to do and hand in*, and deliberately contain very little explanation. All of the background (how a language model works, what perplexity is, why the shadow-model attack from the lecture does not scale to large language models, how each attack is defined, and how to obtain valid membership labels) is in Appendix A. Read Appendix A.1 to A.6 before starting Section 3.1: most failed submissions fail at the dataset stage, not the implementation stage.

Learning goals

By the end of this question you should be able to:

- explain what a membership inference attack (MIA) is and what threat model it assumes;
- construct a valid member / non-member evaluation set, and explain why most naive constructions are invalid;
- implement a likelihood-based MIA from its paper description;
- evaluate an attack properly and interpret what its score does and does not tell you;
- reason about whether these attacks transfer to real, deployed models.

3.1 Building your evaluation dataset

Choose a dataset from the Hugging Face Hub and turn it into a membership inference benchmark satisfying:

Total size	≈ 10,000 examples (trim if larger; not below 8,000)
Label balance	50/50 member / non-member ($\pm 2\%$)
Ground truth	Route A or Route B of Appendix A.6
Source	Hugging Face Hub, your choice

You may *not* label an example a member because it looks familiar. Membership must follow from how the data was constructed. Appendix A.6 gives the two legitimate routes; state clearly which one you took.

Groups with attack ID 0: reserve an additional pool of non-member texts, disjoint from your 10,000 evaluation examples. Your attack needs it; see Appendix A.7.

Question 3.1

How can you *guarantee*, rather than merely hope, that your member and non-member sets are balanced and drawn from the same distribution? Describe a construction procedure that makes this true by design rather than by inspection after the fact. What properties of the source corpus does your procedure rely on?

Hand in (15%). At most one page: which dataset and route you chose, how you sampled it, the size of each side, how you enforced the balance, and your answer to Question 3.1.

3.2 Validating your split

This part carries the most marks. A membership inference benchmark only measures membership if members and non-members come from the same distribution. Appendix A.6 explains why, and what happens when they do not.

Provide evidence that your two sides are comparable along at least these axes:

1. **Length.** Token-count distributions for both sides: plot, report mean and median, and give a two-sample test.
2. **Time.** If your texts carry dates or discuss datable events, show that both sides span the same period.
3. **Domain and source.** Confirm both sides come from the same subset of the corpus, and state how you confirmed it.
4. **Surface statistics.** Vocabulary overlap, type–token ratio, and the frequency of structural markers (markdown, code, boilerplate).

Blind baseline (required). Train a classifier — TF–IDF features plus logistic regression is sufficient — to predict the member / non-member label *without using Pythia at all*, and report its AUC-ROC.

- $\text{AUC} \approx 0.5$: the two sides are not separable from surface text alone, so whatever signal your MIA finds later is plausibly about the model.
- $\text{AUC} \gg 0.5$: your split is confounded and your MIA number will be inflated by an amount you cannot measure. Either fix the split (resample, length-match, restrict the time window) or report both numbers side by side and discuss the gap.

Question 3.2

Suppose a colleague reports an MIA with AUC 0.85 on their own benchmark. What would you ask to see before believing that the model memorised their data? List the checks in the order you would run them, and say what result would falsify the claim.

Hand in (30%). Plots, statistics, blind-baseline AUC, any corrective resampling you performed, and your answer to Question 3.2.

3.3 Implementing your assigned attack

Compute `attack_id = group_number % 4`:

ID	Attack	Reference
0	ReCaLL	Xie et al., EMNLP 2024
1	Reference model (calibrated loss)	Carlini et al., USENIX Security 2021
2	Min-K% Prob	Shi et al., ICLR 2024
3	Min-K%++	Zhang et al., ICLR 2025

Every group additionally implements the LOSS attack (Yeom et al., CSF 2018). It is a few lines of code, it is the baseline every paper compares against, and without it your assigned attack’s number cannot be interpreted.

Definitions, formulas and implementation caveats are in Appendix A.7, or you can refer to the respective reference paper for the details. Read the entry for your attack in full before writing code; ID 0 in particular imposes a constraint on how you built your dataset.

Hand in (20%). A runnable script or notebook, plus a paragraph explaining in your own words *what your attack controls for* — what component of the raw loss it removes, or what it perturbs — *and why that helps*.

3.4 Evaluation

Report, for both LOSS and your assigned attack:

1. **AUC-ROC**, with the ROC curve plotted.
2. **Accuracy**. Accuracy needs a threshold; AUC does not. Hold out a calibration split, choose the threshold there, and report accuracy on the remainder. State your procedure. Choosing the threshold on the same data you evaluate on is a methodological error and will cost marks.
3. **TPR at 5% FPR**. An attack that is right on average but never confidently right about any individual is not much of a privacy threat.
4. Your **blind-baseline AUC** from Section 3.2, in the same table.

Hand in (20%). Results table, ROC curves.

3.5 From Pythia to the real world

Everything above relied on a convenience that does not exist in practice: you knew the training corpus.

Question 3.4

Can the attack you implemented be run against a deployed, commercially available model? Work through what would change, addressing at least:

- **Ground truth**. Without a published training corpus, how would you obtain non-members you are confident about? How does your answer to Question 3.1 apply?

- **Access.** Your attack needs per-token log-probabilities. Which are actually exposed by commercial APIs today, and at what granularity? What remains possible if you receive only generated text?
- **Scale and training regime.** Frontier models are trained on deduplicated data for roughly one epoch over trillions of tokens. What does each of those facts do to the membership signal?
- **Cost.** What does one AUC number cost, in queries?
- **Attack-specific assumptions.** Does your assigned attack require anything of the attacker beyond query access? Does that requirement survive contact with a deployed model?

Then propose concrete ways to overcome these obstacles, stating for each what it buys you and what it gives up. Finally: is a *negative* MIA result on a deployed model evidence that the model is private? Justify your answer.

Hand in (15%). answer to the question.

Marking

Percentages are shares of the marks for Section 3. Every group member must be able to explain any part of the submission.

	Item	Weight
1	Dataset construction, Question 3.1	15%
2	Split validation, blind baseline, Question 3.2	30%
3	Attack implementation	25%
4	Evaluation results	15%
5	Public-model discussion, Question 3.4	15%

On negative results. A careful attack achieving AUC 0.51 with a clear explanation is a better submission than one reporting AUC 0.85 without checking for confounds. We are looking for the check.

A Background: membership inference on language models

Background for Section 3. A membership inference attack answers one question: **was this specific piece of text part of the model’s training data?**

This matters because membership is itself private information. Knowing that a particular clinical note, private email, or copyrighted book was in a training corpus is a disclosure, even if the text is never reproduced. MIA is also the standard building block for auditing: it is how researchers, regulators and plaintiffs try to *prove* that particular data was used.

A.1 What a language model computes

An autoregressive language model factorises the probability of a text into a product of next-token predictions:

$$p_{\theta}(x) = \prod_{t=1}^n p_{\theta}(x_t | x_{<t}).$$

At each position the model outputs a distribution over the whole vocabulary, conditioned on everything before it. Generation is repeated sampling from that distribution.

Training minimises the negative log-likelihood of the training corpus:

$$\mathcal{L}(\theta) = -\frac{1}{|D|} \sum_{x \in D} \sum_{t=1}^n \log p_{\theta}(x_t | x_{<t}).$$

Read that objective carefully, because the whole field follows from it. **The model’s loss on a text is not a neutral property of that text — it is precisely the quantity gradient descent was pushing downward on every training example.** Members had their loss explicitly optimised; non-members did not. Whatever residue of that optimisation survives is the signal an attacker hunts for.

A.2 Perplexity

Perplexity reports that loss on an interpretable scale:

$$\text{PPL}(x) = \exp\left(-\frac{1}{n} \sum_{t=1}^n \log p_{\theta}(x_t | x_{<t})\right) = \exp(\text{average loss}).$$

Intuitively it is an *effective branching factor*: a perplexity of 12 means the model was on average as uncertain as if choosing uniformly among 12 options at each token. Perplexity 1 means perfect prediction; perplexity equal to the vocabulary size means the model learned nothing.

Two properties will follow you through the whole question:

- **It is tokenizer-dependent.** Perplexity values are not comparable across models with different vocabularies.

- **It conflates two different things.** Perplexity is low when a text is *familiar* — and also when a text is simply *easy*. Legal boilerplate, repetitive code and formulaic news openings all have low perplexity under a model that has never seen them. This confusion is the central difficulty of MIA on language models, it is what Section 3.2 asks you to control for, and it is what the attacks in Appendix A.7 are trying to correct.

A.3 The attack from the lecture: shadow models

In the lecture you met the classical formulation (Shokri et al., 2017). The attacker cannot see inside the target model, so instead they *learn* what membership looks like:

1. Train k **shadow models** on data drawn from a distribution similar to the target’s training set, with a similar architecture and training procedure.
2. For each shadow model the attacker knows exactly which examples were in and which were out. Record the model’s behaviour (confidence vector, loss) on both.
3. That yields a labelled dataset of the form (*behaviour*, *IN/OUT*). Train a **meta-classifier** on it.
4. Apply the meta-classifier to the target model’s behaviour on the example in question.

This is powerful because it does not require guessing a threshold, and because it calibrates per example: some examples are intrinsically hard and have high loss whether or not they were trained on. LiRA (Carlini et al., 2022) is the modern, sharper version of the same idea.

A.4 Why this does not work for LLMs

The recipe assumes you can train many models like the target. For large language models that assumption collapses:

- **Cost.** Shadow models must match the target’s scale and training procedure to be informative. Pythia-1B is a billion parameters trained on hundreds of billions of tokens, and a LiRA-quality attack wants tens to hundreds of shadow models. That is tens to hundreds of full pretraining runs, for one attack, on one model.
- **Data.** You would need many independent draws from the same distribution as the training corpus. For web-scale pretraining, “the distribution” is close to *all public text*; you cannot resample it in any meaningful way.
- **Training regime.** Classical MIA relies on overfitting from many epochs over a small dataset. LLMs typically see most data once, so the gap between member and non-member behaviour is correspondingly tiny.
- **What is a record?** Documents are concatenated and chunked into fixed-length training sequences, so the boundary of a training example is not the boundary of a document. It is not even obvious what unit membership should be defined over.

Shadow models remain viable in one LLM setting: **fine-tuning**. If the target is a small adapter trained on a modest dataset for several epochs, you can afford to replicate it many times. That is why Route B in Appendix A.6 is a genuinely different threat model, not merely a convenience.

A.5 What is used instead: score-based attacks

Because the attacker cannot afford to *learn* the decision rule, they hand-design one. Every attack in this question has the same three-step shape:

1. Run the candidate text through the target model in a **single forward pass** and collect per-token log-probabilities.
2. Aggregate them into a scalar **membership score** $s(x)$, applying some correction.
3. Compare against a threshold: $s(x) > \tau \Rightarrow$ predict *member*.

Step 2 is where all the intellectual content lives. Raw loss confuses *seen* with *easy* (Appendix A.2), so most published attacks are one answer to the question: how do I estimate the “easy” component of this text’s loss and subtract it off?

- **Reference model** estimates it with a second language model’s loss on the same text.
- **Min-K%** sidesteps averaging entirely and looks only at the tokens the model handled worst.
- **Min-K%++** estimates it per token, from the model’s own output distribution at that position.

One attack takes a different route:

- **ReCaLL** does not correct the loss at all. It changes the *context* the text is read in — prefixing it with known non-member text — and asks how far the model’s confidence moves. The premise is that memorised text is disturbed more by an unfamiliar context than unmemorised text is.

Schema

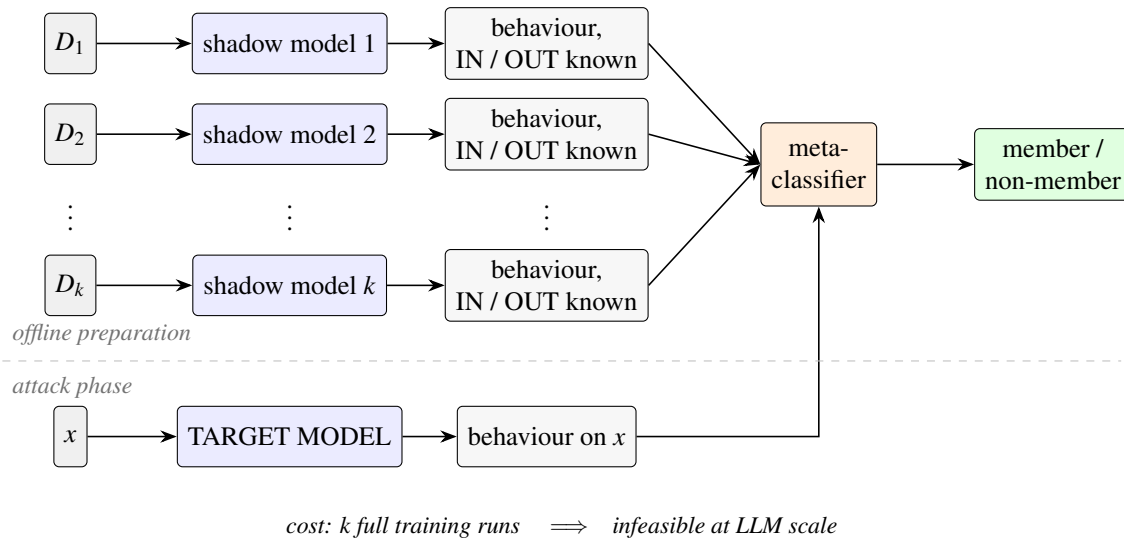


Figure 1: Shadow-model MIA: the formulation from the lecture.

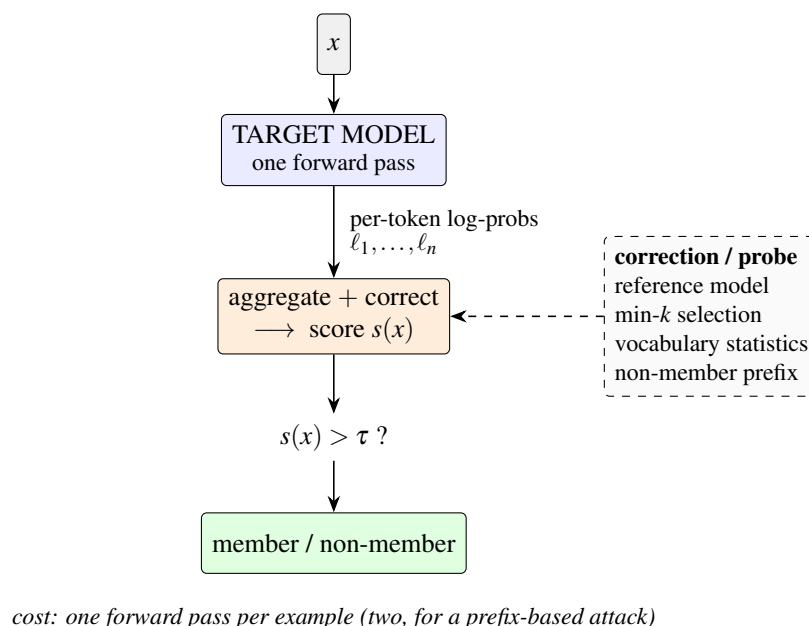


Figure 2: Score-based MIA: what you will implement.

A.6 Constructing a valid benchmark

You need ground truth

You cannot label an example a member because it looks familiar. You need to *know*. Pythia-1B was trained on **the Pile**, so there are only two legitimate routes.

Route A — pretraining membership (default). Members come from the Pile’s *training* split, non-members from its held-out *validation* or *test* split. Both sides then come from the same corpus, collected the same way, at the same time (exactly the property you need). You choose *which domain* to work on (Wikipedia, GitHub, PubMed Central, ArXiv, common crawl, HackerNews, mathematics, ...). Different groups should choose different domains.

Starting points on the Hub, whose availability you should verify yourself, since Pile mirrors move: iamgroot42/mimir, Pile validation/test mirrors, swj0419/WikiMIA.

If you use WikiMIA, be warned: its members and non-members are separated *by date*. Section 3.2 asks you to detect exactly this kind of problem, and you should expect to find it.

Route B — fine-tuning membership (ambitious groups only). Take any Hugging Face dataset you like and split it in half. LoRA-fine-tune Pythia-1B on one half; that half is now member data by construction, and the other half is non-member. Ground truth is perfect and you control the number of epochs.

Route B lets you study things Route A cannot (for example how membership signal grows with repetition) but you must budget for the training run, and your results will *not* be comparable to Route A groups, because fine-tuned models memorise far more aggressively than pretrained ones.

Why splits go wrong

A benchmark is only measuring membership if members and non-members are drawn from the same distribution. If your members are poetry and your non-members are news articles, any attack will appear to succeed brilliantly (but it is detecting *poetry*, not *membership*). Recall from Appendix A.2 that perplexity is low both for familiar text and for intrinsically predictable text: a model that has never seen a single word of your member set will still score it differently if it is written differently.

This failure mode is not hypothetical. Several published MIA results on LLMs have been shown to be partly or wholly explained by distribution shift between the two sides rather than by memorisation. Length is the most common culprit, followed by time.

The blind baseline required in Section 3.2 exists to catch this. If a classifier with no access to the model can separate your two sides from surface text alone, then so can any perplexity-based score, and you cannot attribute your AUC to memorisation.

A.7 Attack definitions

Notation: for a sequence $x = (x_1, \dots, x_n)$, let $\ell_t = \log p(x_t \mid x_{<t})$ under Pythia-1B, and let $\text{LL}(x) = \frac{1}{n} \sum_t \ell_t$ denote the average log-likelihood (a negative quantity).

LOSS — implemented by every group.

$$s_{\text{LOSS}}(x) = \frac{1}{n} \sum_{t=1}^n \ell_t$$

Higher (that is, lower loss) suggests member. Yeom et al., CSF 2018.

ID 0 — ReCaLL. Build a prefix P by concatenating several texts you know to be **non-members**, then score

$$s_{\text{ReCaLL}}(x) = \frac{\text{LL}(x \mid P)}{\text{LL}(x)}.$$

The empirical finding the method rests on is that conditioning on a non-member prefix depresses the log-likelihood of *member* text more than that of non-member text, which leaves members with the higher ratio. Three things to be careful about:

- **Signs.** Both terms are negative, so the ratio behaves counterintuitively. Work through a numeric example by hand before trusting your implementation.
- **Leakage.** The prefix requires known non-members, and those texts must be held out from your evaluation set. If you build P from examples you later score, you have leaked label information and your AUC means nothing. Reserve a disjoint pool in Section 3.1 and report its size.

- **Shots.** The number of texts in the prefix matters. Sweep it.

Note what this attack assumes: an adversary who already holds confirmed non-member data. Carry that into Question 3.4. Xie et al., EMNLP 2024.

ID 1 — Reference model (calibrated loss). Score by the difference between the target model’s loss and a reference model’s loss on the same text. Your choice of reference model is a design decision you must justify: a smaller Pythia is also trained on the Pile and therefore also a “member model,” which weakens the calibration. Consider a model trained on different data (e.g. GPT-2) and discuss the trade-off. Carlini et al., USENIX Security 2021.

ID 2 — Min-K% Prob. Instead of averaging over all tokens, take the $k\%$ of tokens with the lowest ℓ_t and average only those:

$$s_{\text{Min-}k\%}(x) = \frac{1}{|\mathcal{M}|} \sum_{t \in \mathcal{M}} \ell_t, \quad \mathcal{M} = \text{the } k\% \text{ lowest-}\ell_t \text{ positions.}$$

The intuition: a memorised sequence should contain no *very* surprising tokens, while a novel one usually contains a few. Sweep $k \in \{10, 20, 30, 40, 50\}$. Shi et al., ICLR 2024.

ID 3 — Min-K%++. As above, but first normalise each token’s log-probability against the model’s own output distribution at that position:

$$\hat{\ell}_t = \frac{\ell_t - \mu_{x_{<t}}}{\sigma_{x_{<t}}}, \quad \mu_{x_{<t}} = \mathbb{E}_{z \sim p(\cdot | x_{<t})} [\log p(z | x_{<t})],$$

with $\sigma_{x_{<t}}$ the corresponding standard deviation, both taken over the full vocabulary. Then average $\hat{\ell}_t$ over the min- $k\%$ positions. This separates “the model is confident here” from “this token is easy for anyone.” Note the memory cost: you need full-vocabulary logits at every position, so process in chunks. Zhang et al., ICLR 2025.

A.8 Practical notes and reading

- Pythia-1B in fp16 needs roughly 2 GB of GPU memory. Scoring 10,000 sequences of 128–256 tokens is on the order of 10–20 minutes on a free-tier Colab GPU. Min-K%++ is heavier because of the vocabulary-wide statistics; ReCaLL is heavier because of the second, longer forward pass.
- Use EleutherAI/pythia-1b with the default revision and record the revision hash. The Pythia suite also ships pythia-1b-deduped, trained on a globally deduplicated Pile. Duplication is one of the strongest drivers of memorisation, so the two are not interchangeable — scoring both with the same attack is a cheap and informative extra experiment.
- Fix and report your random seeds.
- Truncate all sequences to the same maximum length and state what it is. Length interacts with every attack here.

Optional background.

- Alammr, *The Illustrated Transformer*
- Karpathy, *Let's build GPT: from scratch, in code.*

Optional reads.

- Shokri et al., *Membership Inference Attacks Against Machine Learning Models*, S&P 2017 — the shadow-model formulation from the lecture.
- Carlini et al., *Extracting Training Data from Large Language Models*, USENIX Security 2021 — Section 5 in particular.
- Duan et al., *Do Membership Inference Attacks Work on Large Language Models?*, COLM 2024 — read this one carefully. It will save you from the most common mistake in this question.